Classification

Hands-On Machine Learning (Géron) - Chapter 3

Giacomo Fiumara

Overview

From Regression to Classification

- Supervised learning tasks are often grouped into regression (predicting values) and classification (predicting classes).
- Previous work focused on regression; this chapter turns to classification systems.
- Main running example: handwritten digit classification on the MNIST dataset.

The MNIST Dataset

What Is MNIST?

- MNIST: 70,000 small images of handwritten digits from US Census Bureau employees and high school students.
- Each image is labeled with the digit it represents (0–9).
- Widely used as the "hello world" of machine learning for classification.

Loading MNIST with Scikit-Learn

- Scikit-Learn provides helper functions to fetch standard datasets (real, toy, or synthetic).
- For MNIST, we can use `fetch_openml('mnist_784', as_frame=False)`.
- Returned object is a `Bunch` with entries such as `DESCR`, `data`, and `target`.
- Using `as_frame=False` returns NumPy arrays, which are convenient for image data.

Structure of MNIST Data

- `X` has shape (70000, 784): 70,000 images, each flattened into 784 features.
- Each image is 28×28 pixels; each feature is a pixel intensity from 0 (white) to 255 (black).
- `y` has shape (70000,): labels are digit characters '0'–'9'.

Visualizing a Single Digit

- To view an image, reshape a feature vector to 28×28 and display it with Matplotlib.
- Use a grayscale colormap (e.g., `cmap="binary"`) so 0 is white and 255 is black.
- The first sample in the book appears as a 5, matching its label.

Figure 3-1 – Example MNIST Image

Figure 3-2 – Sample Digit Images

Training and Test Split

- The MNIST dataset from OpenML is already ordered so first 60,000 samples are training, last 10,000 are test.
- Split with slicing: `X_train = X[:60000]`, `X_test = X[60000:]`, and similarly for `y`.
- Training set is shuffled, which helps cross-validation folds be similar and avoids order sensitivity issues.

Training a Binary Classifier

Binary Classification Setup

- Start with a simpler task: detect only one digit, e.g., "5 vs not-5".
- This is a binary classification problem with two classes: positive (5) and negative (non-5).
- Create boolean target vectors: `y_train_5 = (y_train == '5')`, `y_test_5 = (y_test == '5')`.

SGDClassifier for Large Datasets

- Use `SGDClassifier` for efficient training on large datasets and for online learning.
- It processes training instances one at a time, updating parameters incrementally.
- Train on the full training set: `sgd_clf.fit(X_train, y_train_5)`.
- Then call `sgd_clf.predict([some_digit])` to check if the digit is predicted as 5.

Performance Measures

Using Cross-Validation for Accuracy

- Evaluate classifier with k-fold cross-validation using `cross_val_score`.
- Example: 3-fold cross-validation on the 5-detector yields accuracies around 95–96%.
- Accuracy: proportion of correct predictions among all predictions.

Why Accuracy Can Be Misleading

- MNIST is imbalanced for the "5" class: only about 10% of images are 5s.
- A dummy classifier that always predicts "non-5" reaches about 90% accuracy.
- High accuracy alone does not guarantee a useful classifier on skewed datasets.

Implementing Cross-Validation Manually

- `StratifiedKFold` creates stratified folds with similar class proportions in each fold.
- For each split, clone the classifier, fit on training folds, and evaluate on the validation fold.
- This gives a clear view of fold-wise performance and full control over the process.

Confusion Matrix Concept

- A confusion matrix counts how often instances of each actual class are predicted as each possible class.
- Binary case: rows are actual classes (negative / positive), columns are predicted classes.
- Entries: true negatives, false positives, false negatives, and true positives.

Computing a Confusion Matrix

- Use `cross_val_predict` to obtain out-of-sample predictions for every training instance.
- `confusion_matrix(y_train_5, y_train_pred)` returns a 2×2 matrix.
- Example counts: many true negatives, some false positives, some false negatives, and thousands of true positives.

Interpreting the Confusion Matrix

- First row (actual non-5): entries show correct rejections (true negatives) and mistaken "5" predictions (false positives).
- Second row (actual 5): entries show missed 5s (false negatives) and correctly detected 5s (true positives).
- A perfect classifier has nonzero values only on the main diagonal.

Figure 3-3 – Illustrated Confusion Matrix

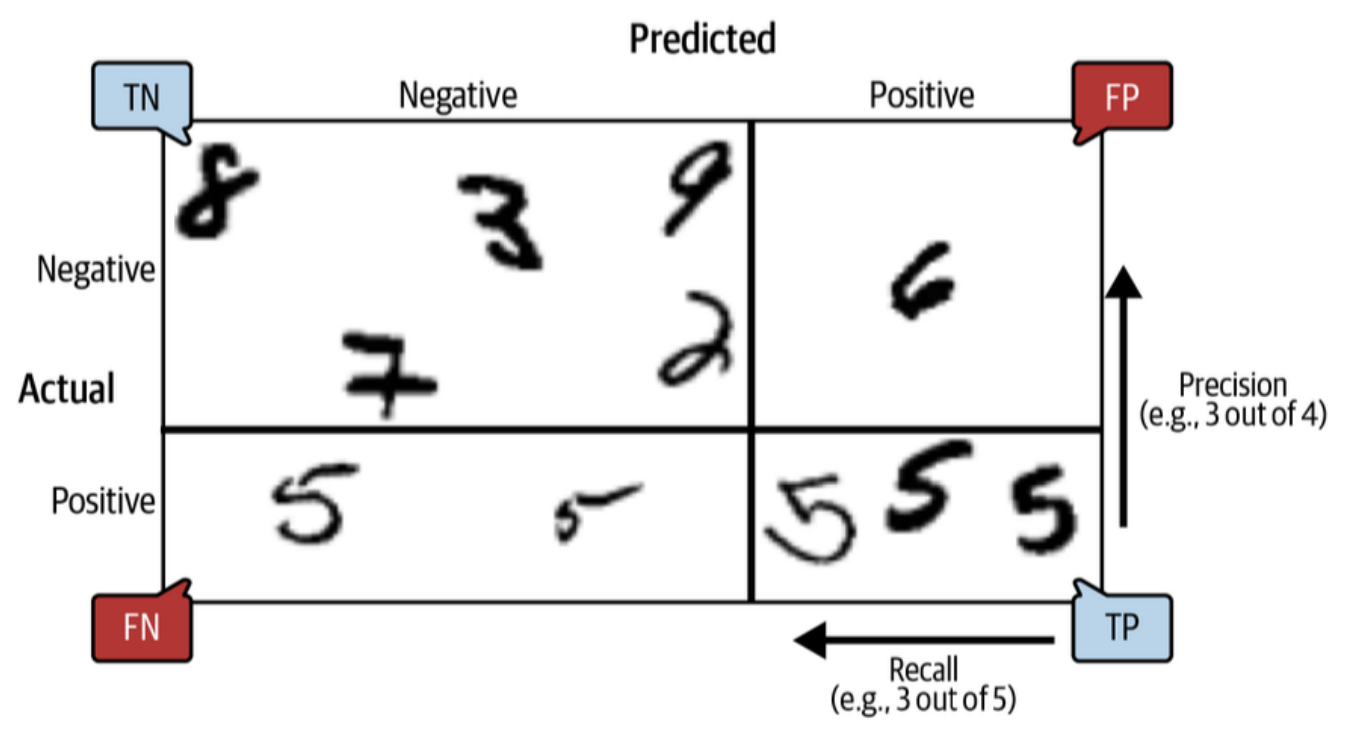


Figure 1: Figure 3-3: Visual illustration of true/false positives and negatives.

Precision and Recall Definitions

- Precision: among all instances predicted positive, fraction that are actually positive (TP / (TP + FP)).
- Recall (sensitivity or true positive rate): among all actual positives, fraction correctly predicted positive (TP / (TP + FN)).
- Both metrics rely on the confusion matrix counts.

Computing Precision and Recall

- In Scikit-Learn, use `precision_score(y_true, y_pred)` and `recall_score`.
- For the 5-detector example: precision is about 83.7%, recall about 65.1%.
- Compared to raw accuracy, these give a clearer picture of positive-class performance.

F1 Score

- F1 score: harmonic mean of precision and recall, high only when both are high.
- Formula: $F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$.
- Useful when a single number is needed to compare classifiers, but it implicitly balances precision and recall equally.

Choosing Precision vs Recall

- Some applications prefer high precision (few false positives), even at the cost of lower recall.
- Others prefer high recall (few false negatives), accepting more false positives.
- Examples: child-safe content detection (favor precision), shoplifter detection (favor recall).

Decision Scores and Thresholds

- Classifiers like `SGDClassifier` compute a decision score for each instance before thresholding.
- Default threshold: zero; scores above this are predicted as positive, below as negative.
- Raising the threshold tends to increase precision but decrease recall; lowering it does the opposite.

Figure 3-4 – Precision/Recall Trade-off

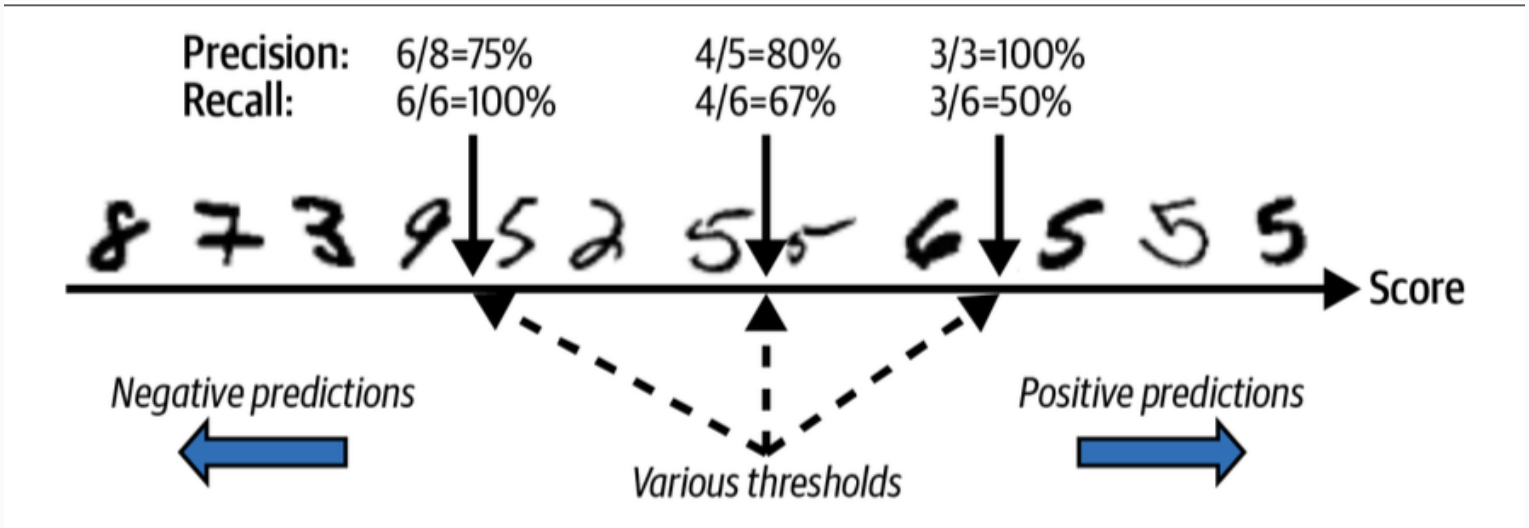


Figure 2: Figure 3-4: Changing the decision threshold shifts precision and recall.

Precision and Recall vs Threshold

- Use `cross_val_predict` with `method="decision_function"` to get scores for all training instances.
- `precision_recall_curve` returns precision, recall, and threshold arrays across possible cutoffs.
- Plotting precision and recall against thresholds helps visualize trade-offs.

Figure 3-5 – Precision/Recall vs Threshold

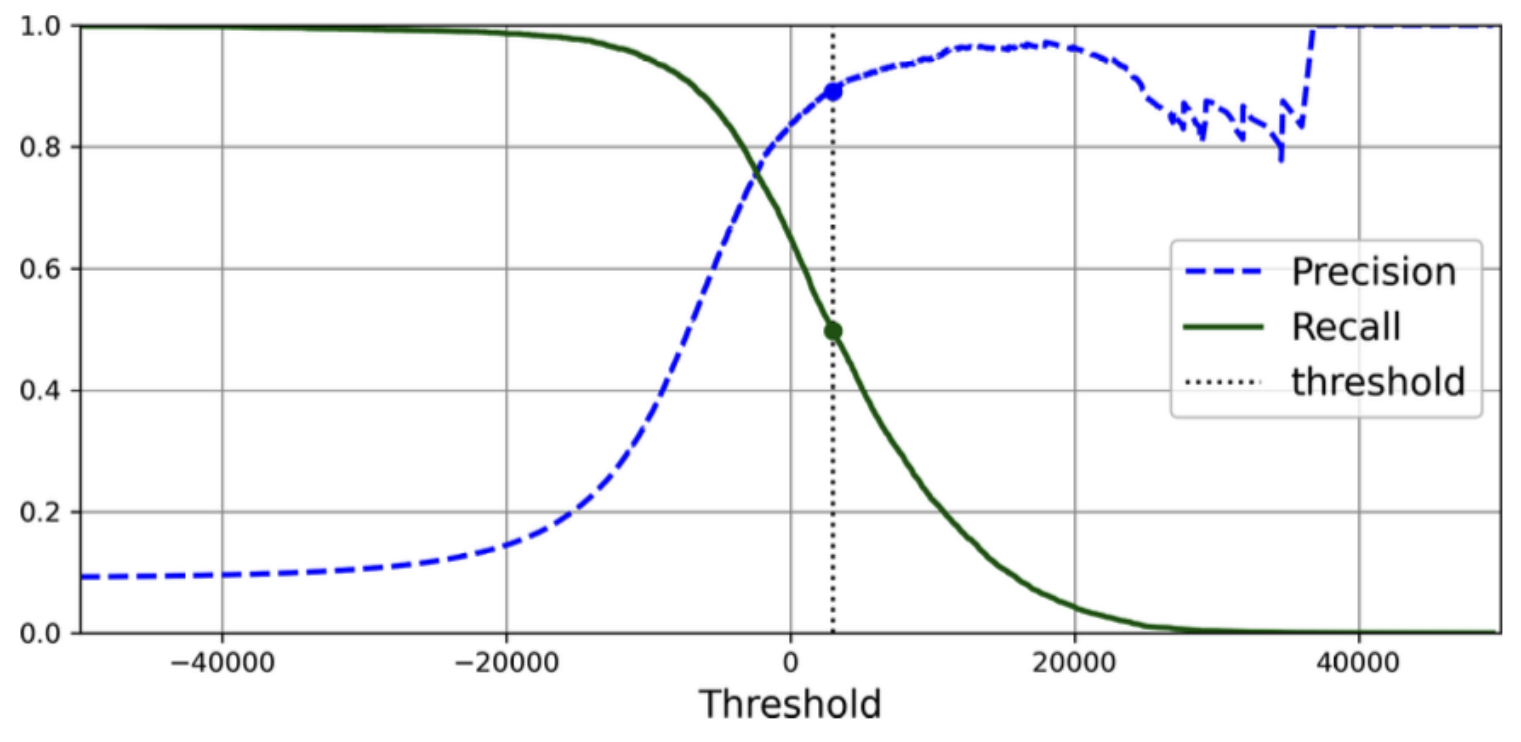


Figure 3: Figure 3-5: Precision and recall as functions of the decision threshold.

Precision vs Recall Curve

- Plotting precision directly against recall shows how one changes as the other is improved.
- The goal is a curve close to the top-right corner (high precision and high recall).
- A chosen operating point (e.g., 90% precision) corresponds to one specific threshold.

Figure 3-6 – Precision vs Recall

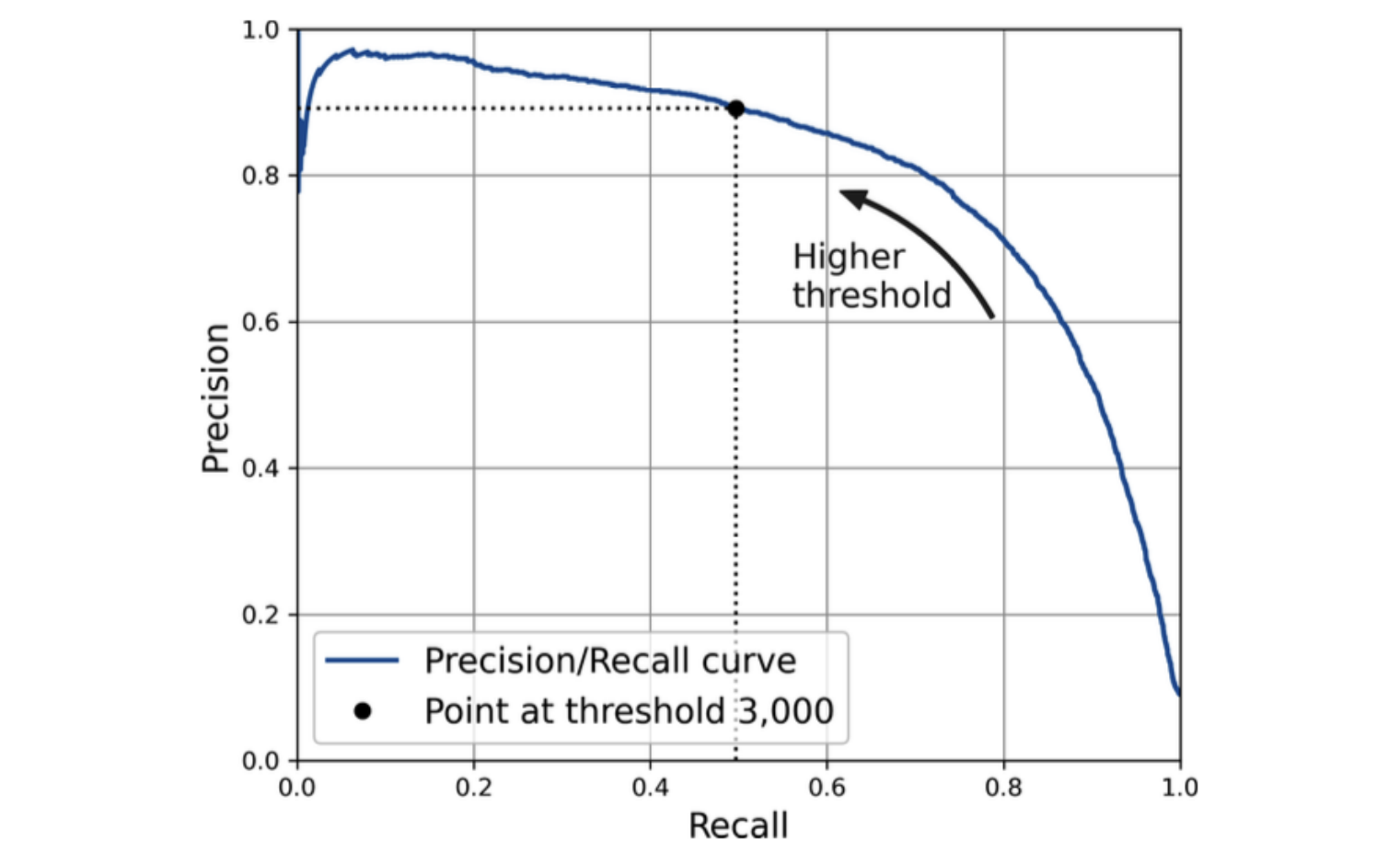


Figure 4: Figure 3-6: Precision as a function of recall with a highlighted operating point.

Tuning for Target Precision

- To reach a target precision (e.g., 90%), find the smallest threshold where precision exceeds this level.
- Once the threshold is fixed, you can recompute recall at that threshold.
- Higher precision typically comes with reduced recall, so both should be reported.

ROC Curve Basics

- Receiver operating characteristic (ROC) curve plots true positive rate (recall) vs false positive rate.
- False positive rate: fraction of negative instances incorrectly predicted positive (FP / (FP + TN)).
- ROC visualizes trade-off between sensitivity (recall) and fall-out (1 - specificity).

Computing ROC and AUC

- Use `roc_curve(y_true, y_scores)` to get FPR, TPR, and thresholds.
- Plot FPR vs TPR, along with the diagonal line representing a random classifier.
- ROC AUC (area under curve) summarizes performance: 1.0 is perfect, 0.5 is random.

Figure 3-7 – ROC Curve

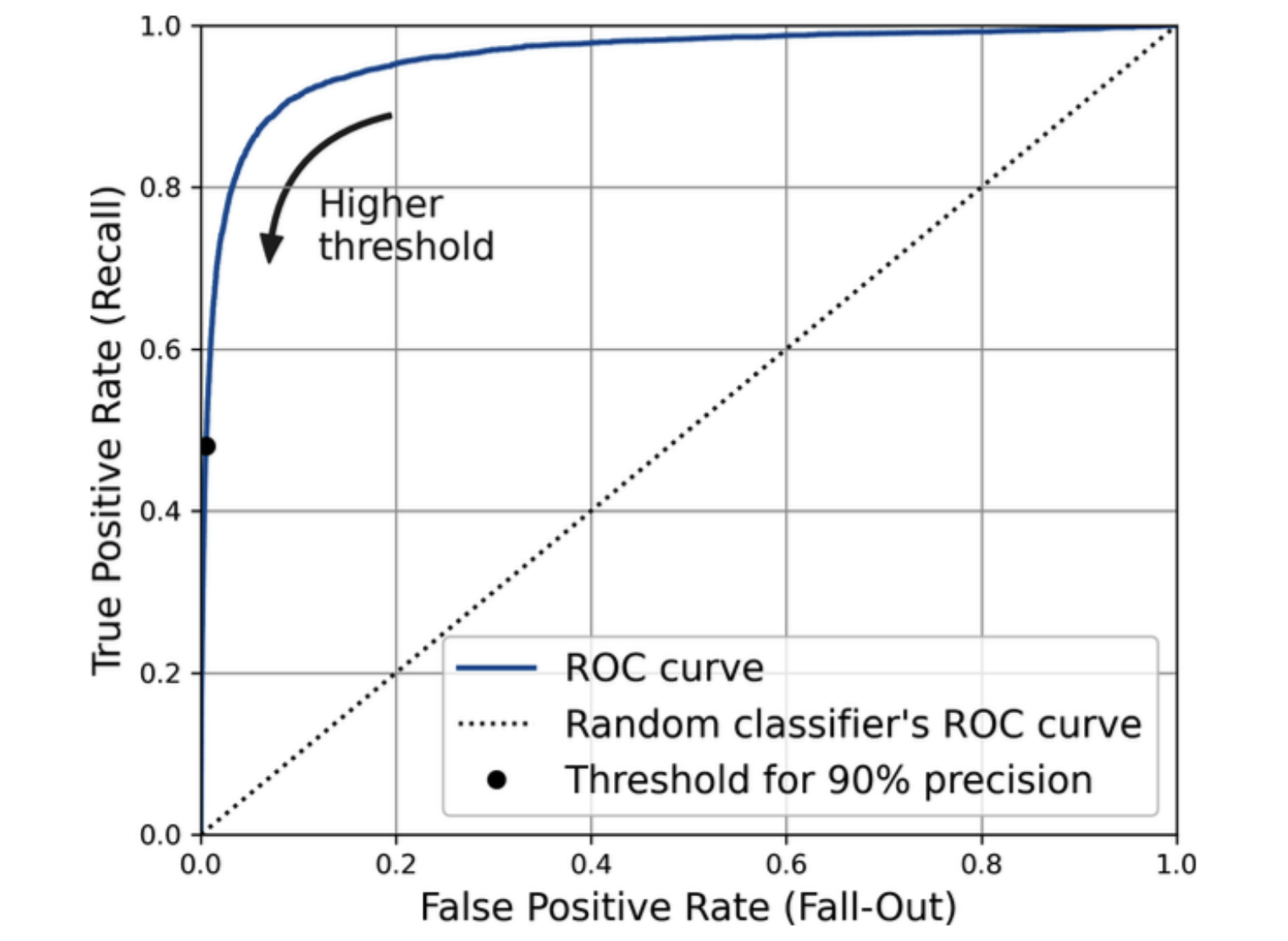


Figure 5: Figure 3-7: ROC curve with a specific threshold highlighted.

Choosing Between PR and ROC Curves

- PR curves are preferable when the positive class is rare or when false positives are particularly costly.
- ROC curves can look deceptively good in highly imbalanced settings.
- For the 5-detector, the PR curve reveals more room for improvement than the ROC curve alone.

Random Forest vs SGD – Scores

- `RandomForestClassifier` can also be evaluated via PR and ROC curves.
- Use `predict_proba` to obtain class probabilities and treat positive-class probabilities as scores.
- Comparing curves: Random forest's PR curve lies closer to top-right and its ROC AUC is near 1, indicating superior performance.

Figure 3-8 – Comparing PR Curves

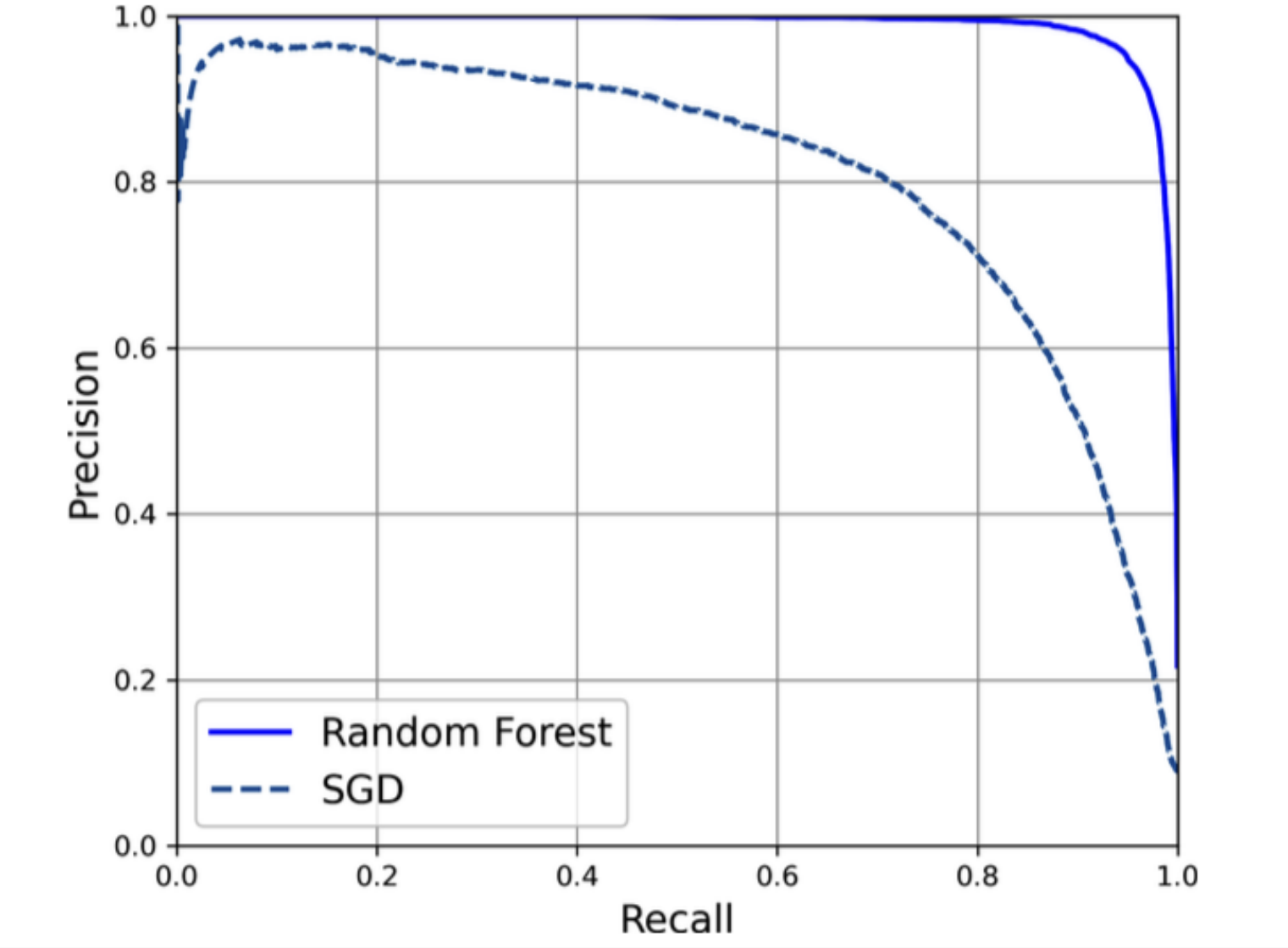


Figure 6: Figure 3-8: PR curves for SGD and random forest classifiers.

Multiclass Classification

From Binary to Multiclass

- Binary classifiers distinguish between two classes; multiclass classifiers handle three or more.
- Some algorithms (e.g., logistic regression, random forest, naive Bayes) are inherently multiclass.
- Others (e.g., SVM, SGD) are binary but can be combined to handle multiple classes.

OvR and OvO Strategies

- One-vs-rest (OvR): train one binary classifier per class; choose the class whose classifier outputs the highest score.
- One-vs-one (OvO): train one classifier for every pair of classes; classification is by majority vote across all pairwise decisions.
- OvO needs $N(N-1)/2$ classifiers for N classes; each sees fewer examples.

Automatic Multiclass Handling in Scikit-Learn

- Scikit-Learn automatically wraps binary classifiers in OvR or OvO strategies when used on multiclass targets.
- Example: `SVC` on MNIST uses OvO by default (45 classifiers for 10 digits).
- Decision scores: one per class, reflecting number of "wins" across pairwise classifiers.

Manual Control with Wrapper Classes

- `OneVsRestClassifier` and `OneVsOneClassifier` let you explicitly choose the strategy.
- They wrap any base estimator (binary or multiclass) and expose a standard multiclass interface.
- Each wrapper stores the individual estimators in an attribute such as `estimators_`.

Evaluating Multiclass SGD on MNIST

- Train `SGDClassifier` on all 10 digit classes and evaluate with cross-validation.
- Accuracy around 86–87% before scaling; about 89–90% after feature scaling.
- Scaling inputs (e.g., with `StandardScaler`) is important for gradient-based methods.

Error Analysis

Why Analyze Errors?

- Once a promising model is selected, error analysis helps identify directions for improvement.
- Examining which classes are confused suggests data collection and feature engineering strategies.
- Visual tools such as confusion matrix plots are especially helpful here.

Multiclass Confusion Matrix

- Use `cross_val_predict` to get predictions, then plot with `ConfusionMatrixDisplay.from_predictions`.
- Non-normalized matrix shows raw counts; diagonal elements represent correct classifications.
- For MNIST, most mass lies on the diagonal, indicating good performance.

Figure 3-9 – Confusion Matrices

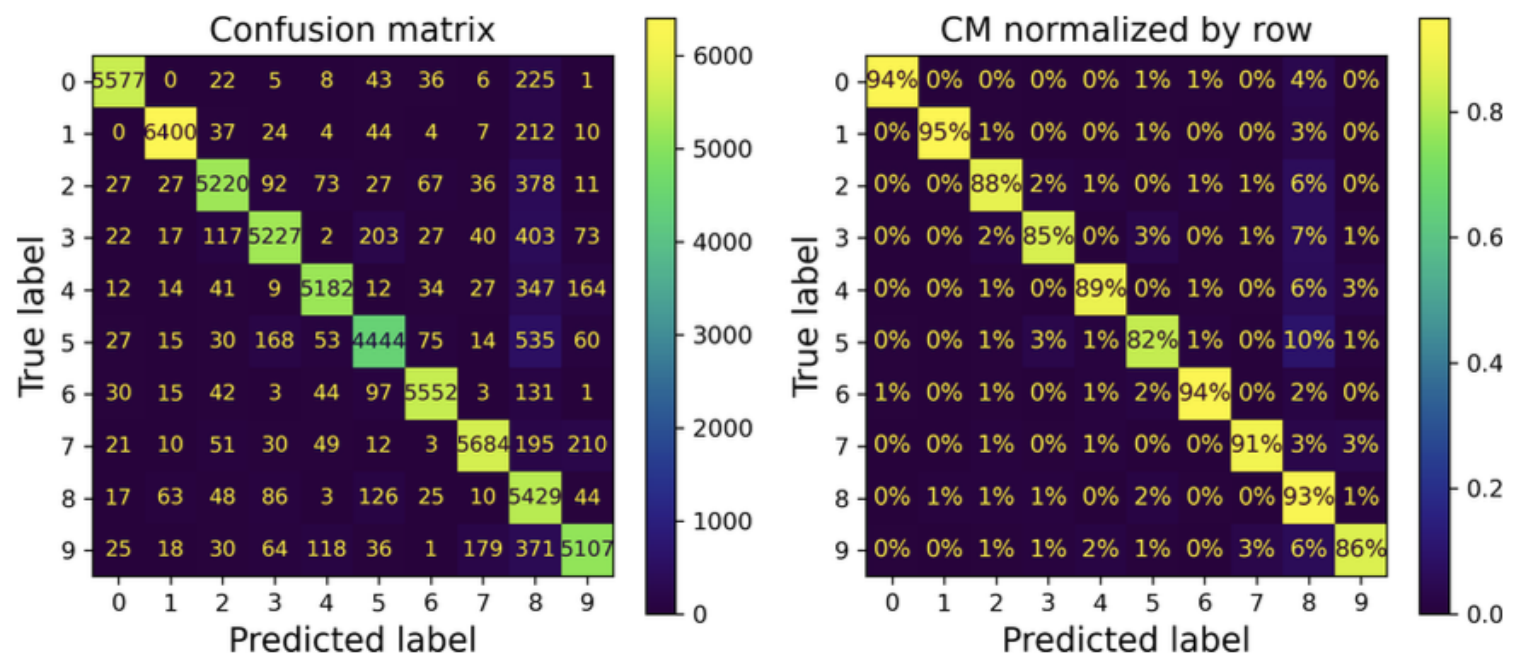


Figure 7: Figure 3-9: Raw confusion matrix (left) and row-normalized matrix (right).

Normalizing the Confusion Matrix

- Row normalization (normalize="true") divides counts by number of instances in each true class.
- This makes it easier to compare per-class accuracy and error rates.
- For example, only about 82% of 5s are correctly classified; many errors are confusions with 8s.

Highlighting Errors Only

- Set sample weights to ignore correct predictions and emphasize only misclassifications.
- Alternatively, normalize by column (normalize="pred") to see, for each predicted class, what the true classes are.
- These views help understand which wrong predictions are most common and how serious they are.

Figure 3-10 – Error-Focused Confusion Matrices

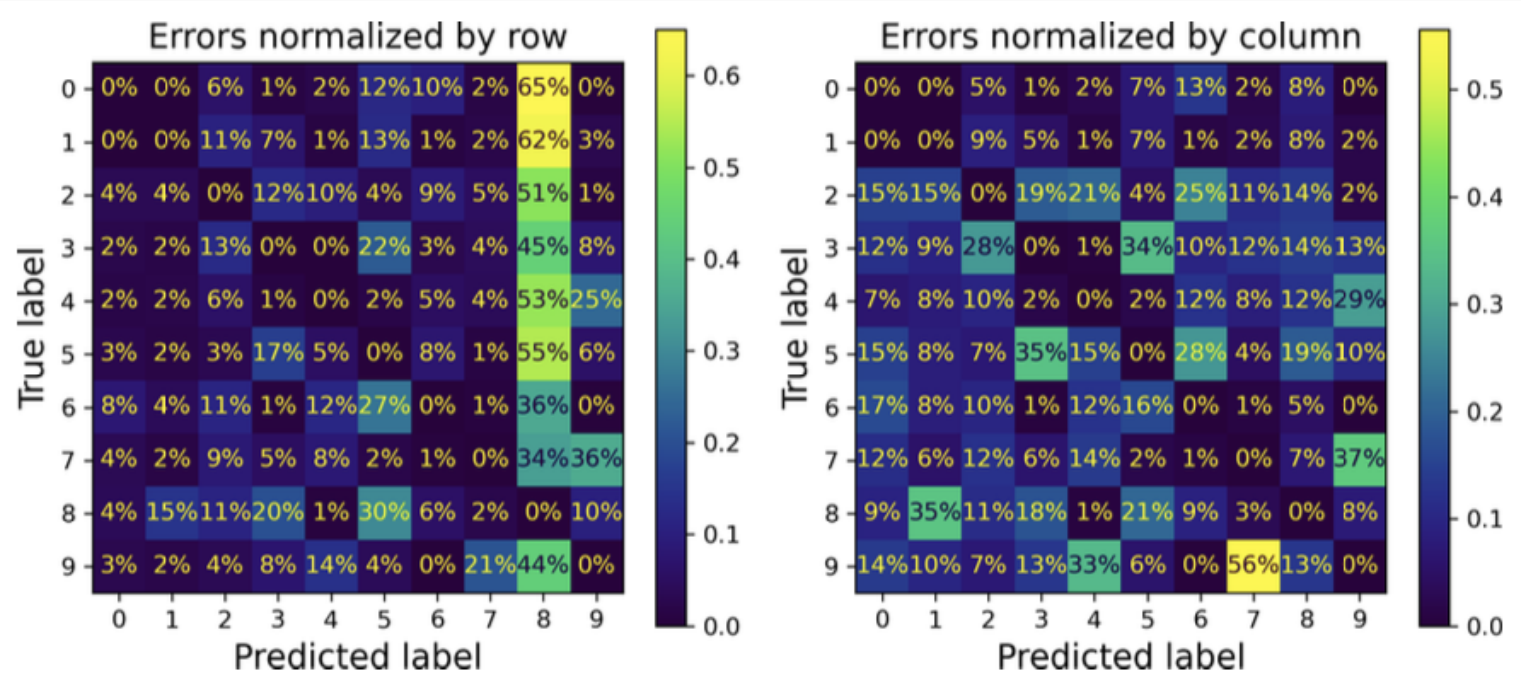
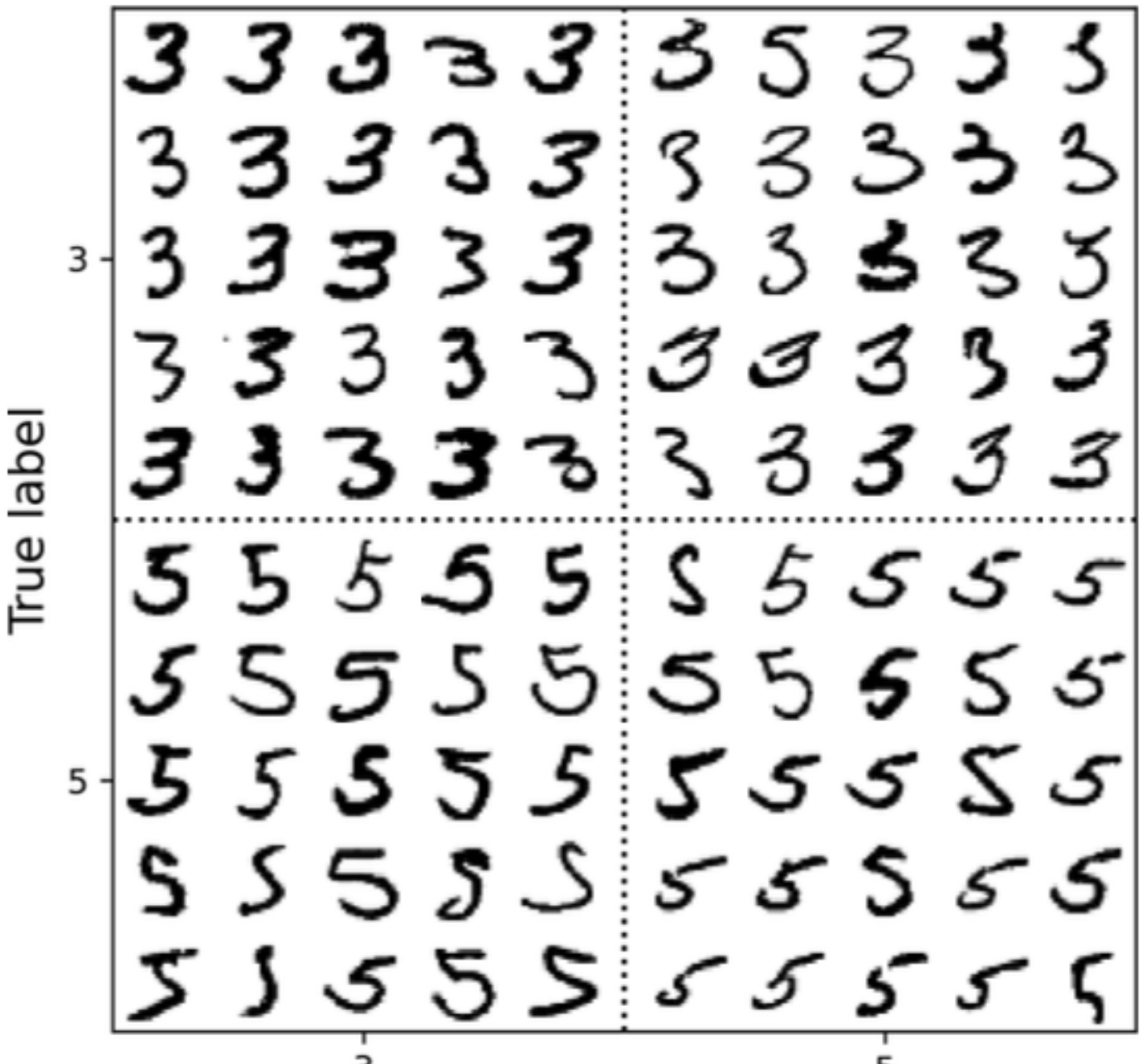


Figure 8: Figure 3-10: Confusion matrices plotting only misclassifications (row- and column-normalized).

Inspecting Specific Confusions

- Focus on pairs of confusing classes (e.g., 3 and 5) by plotting sample images for each combination of true/predicted label.
- Some errors correspond to ambiguous digits that are hard even for humans; others are obvious and suggest model limitations.
- A linear classifier can easily confuse digits that differ only in a few pixels or that are shifted/rotated.

Figure 3-11 – 3 vs 5 Confusion Examples



Ideas for Improving the Classifier

- Collect more training data for commonly confused digit pairs (e.g., digits that look like 8s but are not).
- Engineer features that capture discriminative properties (e.g., number of closed loops).
- Preprocess images to center digits and reduce rotation, or use data augmentation to make the model robust to shifts and distortions.

Multilabel Classification

What Is Multilabel Classification?

- In multilabel classification, each instance can be assigned multiple labels simultaneously.
- Example: face recognition in a group photo where several known people may appear at once.
- Output is a vector of booleans indicating presence/absence of each label.

Simple Multilabel Example on MNIST

- Define two labels per digit: "large" (digit ≥ 7) and "odd" (digit is 1,3,5,7,9).
- Build `y_multilabel` with two columns, one for each of these properties.
- Train a `KNeighborsClassifier`, which supports multilabel targets.

Evaluating Multilabel Classifiers

- Metrics must now consider multiple labels; one approach is to compute F1 for each label and average.
- `f1_score(..., average="macro")` gives an unweighted average across labels.
- `average="weighted"` uses label supports as weights, emphasizing frequent labels.

Capturing Label Dependencies

- Independent binary classifiers for each label ignore correlations between labels.
- In some problems, one label's presence affects another's probability (e.g., "large" and "odd" digits).
- Classifier chains address this by feeding earlier label predictions as additional features to later models in the chain.

Using ClassifierChain

- `ClassifierChain` wraps a base estimator (e.g., `SVC`) to model label dependencies.
- If `cv` is set, cross-validated out-of-sample predictions are used when training later labels in the chain.
- This approach improves multilabel performance when label relationships are strong.

Multioutput Classification

Multioutput–Multiclass Classification

- Multioutput classification generalizes multilabel classification to multiclass labels.
- Each label position can take more than two possible values (e.g., pixel intensities from 0 to 255).
- Tasks involve predicting complex structured outputs (e.g., entire images).

Digit Image Denoising Example

- Construct noisy versions of MNIST images by adding random noise to pixel intensities.
- Targets are the original clean images, so each pixel is a class with 256 possible intensity values.
- This defines a multioutput task with one label per pixel.

Figure 3-12 – Noisy and Clean Images

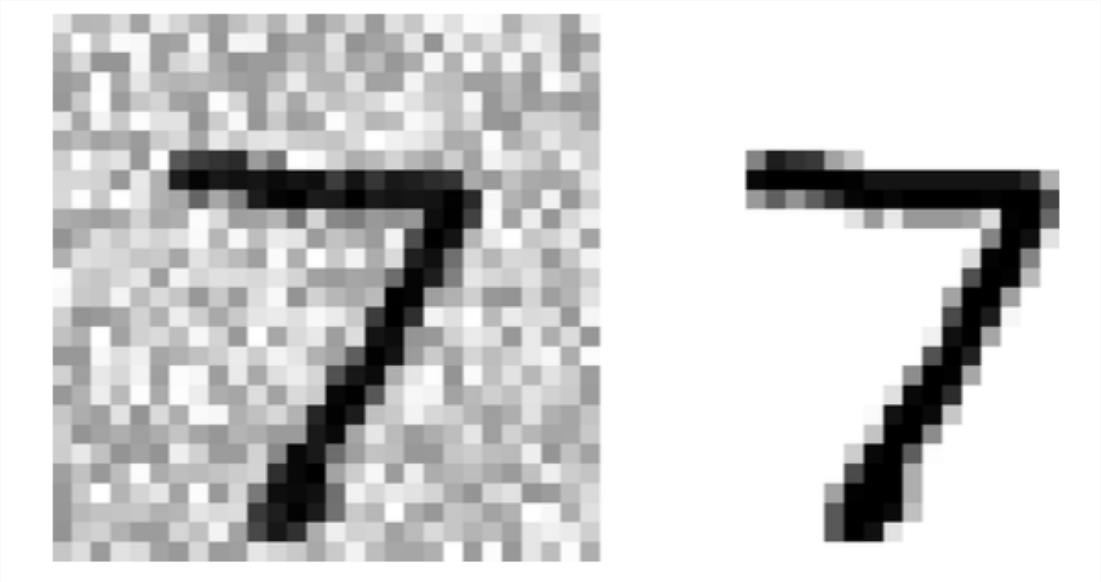


Figure 10: Figure 3-12: Noisy digit image (left) and clean target image (right).

KNN for Denoising

- Train a `KNeighborsClassifier` on noisy images as inputs and clean images as targets.
- At prediction time, for a new noisy image, the classifier returns the clean image corresponding to nearest neighbors.
- This uses multiclass outputs (pixel values) for each output position (pixel).

Figure 3-13 – Cleaned-Up Image

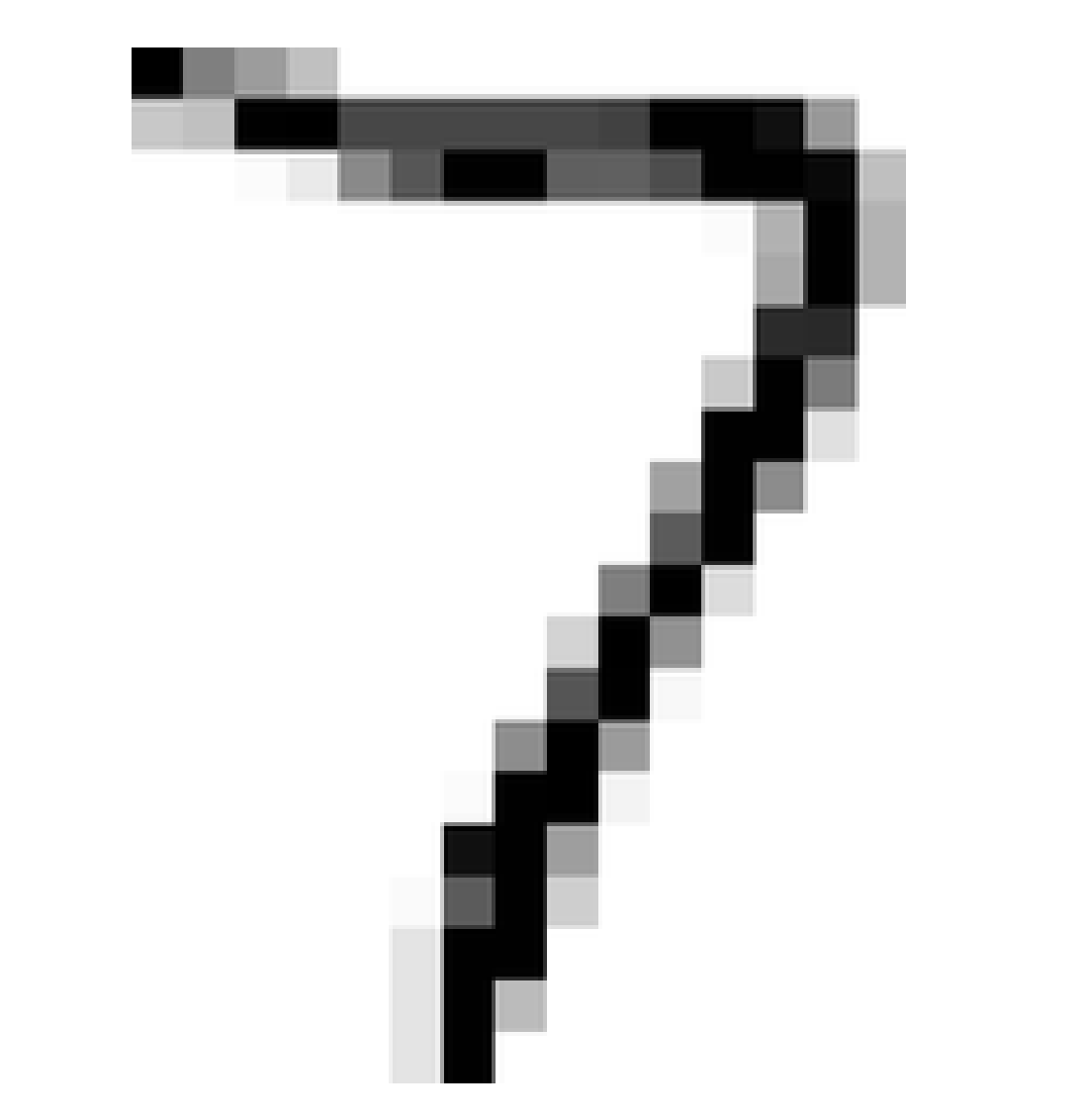


Figure 11: Figure 3-13: Output of the denoising model on a noisy digit

Summary

Key Takeaways

- Classification involves predicting discrete classes, and its evaluation requires richer metrics than accuracy alone.
- Confusion matrices, precision, recall, F1, PR curves, and ROC curves help characterize classifier behavior.
- Techniques extend from binary to multiclass, multilabel, and multioutput scenarios, often via model combinations or wrappers.
- Careful error analysis, feature engineering, data augmentation, and appropriate model choice can substantially improve performance.